

Санкт-Петербургский политехнический университет Петра Великого
Институт машиностроения, материалов и транспорта
Высшая школа автоматизации и робототехники

КУРСОВАЯ РАБОТА

**Разработка игры в жанре «bullet hell» на языке C++ с
использованием библиотеки Raylib**
по дисциплине «Объектно-ориентированное программирование»

Выполнил студент
гр. 3331506/30401

Ковалев П. В.

Руководитель
Доцент

Ананьевский М. С.

«____» _____ 2026г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение	4
Глава 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	6
1.1. Жанр bullet hell: особенности и типовые механики	6
1.2. Обзор существующих игровых движков и библиотек для 2D-игр	6
1.3. Выбор языка программирования и библиотеки	7
1.4. Требования к разрабатываемой игре	8
1.5. Постановка задачи	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ	10
2.1. Общая структура приложения	10
2.2. Объектная модель	10
2.3. Система управления игроком	11
2.4. Система снарядов и коллизий	12
2.5. Система событий и планировщик атак	12
2.6. Рендеринг и преобразование координат	13
ГЛАВА 3. РЕАЛИЗАЦИЯ ИГРОВЫХ МЕХАНИК И УРОВНЕЙ	15
3.1. Реализация базовых механик	15
3.1.1. Движение игрока	15
3.1.2. Коллизии и неуязвимость	15
3.2. Реализация системы событий и планировщика	16
3.2.1. Структура AttackEvent	16
3.2.2. Планировщик в update_level	16
3.2.3. Вспомогательные функции для генерации атак	17
3.3. Создание уровня «Beginning»	17
3.4. Создание уровня «Way»	18
3.5. Режим отладки и вспомогательные функции	18
3.6. Основные трудности и их решение	18
ГЛАВА 4. ТЕСТИРОВАНИЕ И ОТЛАДКА	20
4.1. Цели и подходы к тестированию	20
4.2. Тестовые уровни	20
4.2.1. Уровень TEST	20
4.2.2. Уровень TEST_HP	20
4.2.3. Уровень EMPTY	21
4.3. Тестирование планировщика событий	21
4.4. Тестирование производительности	21
4.5. Режим отладки как инструмент тестирования	22
4.6. Результаты тестирования	22

ГЛАВА 5. СБОРКА И ЗАПУСК	23
5.1. Требования к системе	23
5.2. Установка зависимостей	23
5.3. Получение исходного кода	23
5.4. Сборка с использованием Premake	24
5.5. Запуск игры	24
5.6. Управление в игре	24
5.7. Возможные проблемы и их решение	25
Заключение	27

ВВЕДЕНИЕ

Актуальность работы определяется необходимостью создания лёгких, переносимых и при этом достаточно выразительных игровых движков для обучения основам разработки компьютерных игр. Жанр «bullet hell» предъявляет повышенные требования к производительности и управлению большим количеством динамических объектов (снарядов), что делает его показательной задачей для отработки эффективных алгоритмов обработки данных, коллизий и событийного программирования. Библиотека Raylib, представляющая собой минималистичный, но полнофункциональный API для 2D- и 3D-графики, оконного управления и ввода, позволяет создавать игры без использования тяжёлых коммерческих движков (Unity, Unreal Engine) и при этом сохранять контроль над низкоуровневыми аспектами: циклами обновления, памятью, временем. Таким образом, разработка игры в жанре bullet hell на C++ с использованием Raylib является актуальной учебно-исследовательской задачей.

Новизна работы заключается в разработке событийно-ориентированной системы описания уровней, позволяющей гибко задавать временные последовательности атак (в том числе создавать вложенные события, когда одна атака порождает другие), а также в применении стандарта C++20 (в частности, `std::format` для форматирования строк интерфейса) и интеграции с кроссплатформенной системой сборки Premake/CMake. Такой подход упрощает добавление новых уровней и типов атак без изменения основного игрового цикла.

Цель работы: разработать на языке C++ с использованием библиотеки Raylib игру в жанре bullet hell, поддерживающую несколько уровней с различными паттернами атак и управление игроком с клавиатуры.

Объект исследования: методы организации игрового цикла, событийно-ориентированного планирования атак, обработки коллизий и отрисовки в реальном времени.

Предмет исследования: применение возможностей C++20 (включая стандартную библиотеку, контейнеры, алгоритмы, `std::format`) и библиотеки Raylib для реализации аркадной игры.

Задачи работы:

1. Провести обзор существующих решений и выбрать технологический стек.
2. Спроектировать архитектуру приложения: объектную модель игрока и снарядов, систему событий.

3. Реализовать базовые механики: движение игрока (клавиши WASD/стрелки), нормализацию скорости, проверку столкновений (круг–прямоугольник), учёт неуязвимости после попадания.
4. Разработать событийную систему для описания атак уровней, позволяющую создавать сложные последовательности (линии, лучи, взрывы, веера) и динамически добавлять новые события.
5. Создать два демонстрационных уровня (обучающий и сложный) с использованием этой системы.
6. Провести анализ полученных результатов.

Практическая значимость работы заключается в том, что разработанный код может служить основой для дальнейшего развития (добавления новых типов атак, звукового сопровождения, редактора уровней) или использоваться в учебных целях при изучении C++ и игрового программирования.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1. Жанр **bullet hell**: особенности и типовые механики

Жанр компьютерных игр «bullet hell» характеризуется большим количеством снарядов на экране, сложными и часто симметричными паттернами их движения, а также высокими требованиями к реакции игрока. В отличие от классических аркадных шутеров, где основное внимание уделяется поражению врагов, в bullet hell важнейшим навыком становится уклонение от большого числа одновременно движущихся снарядов.

Типовые механики, встречающиеся в играх этого жанра:

- **Плотные линейные залпы** — снаряды движутся параллельными рядами с небольшим интервалом.
- **Лучи и вереницы пуль** — пули выпускаются последовательно по одной траектории, создавая «нитку».
- **Круговые и веерные залпы** — снаряды разлетаются из центра под равными углами.
- **Прицельные снаряды** — пуля направляется в текущее положение игрока на момент выстрела.
- **Бомбы (медленные крупные снаряды)** — после достижения определённой точки взрываются, порождая новые снаряды.
- **Сложные паттерны** — комбинации перечисленных элементов, синхронизированные во времени.

Реализация таких механик требует от разработчика эффективного управления динамическими массивами (постоянное добавление и удаление снарядов), точных и быстрых алгоритмов проверки столкновений, а также удобного способа описания последовательностей атак.

1.2. Обзор существующих игровых движков и библиотек для 2D-игр

Для создания 2D-игр на C++ существует несколько популярных библиотек и фреймворков. Наиболее распространённые из них представлены в таблице 1.1.

Сравнение библиотек для 2D-игр на C++

Библио-тека	Лицензия	Аппаратное ускорение	Кроссплатформенность	Сложность освоения
SFML 2.x	zlib	Да (OpenGL)	Win / Linux / Mac	Низкая
SDL2	zlib	Да (OpenGL / Direct3D)	Win / Linux / Mac / Android / iOS	Средняя
Raylib	zlib	Да (OpenGL)	Win / Linux / Mac / Android / Web	Низкая
GLFW + OpenGL	zlib/libpng	Да (OpenGL)	Win / Linux / Mac	Высокая

SFML предоставляет удобные классы для графики, звука, сети и ввода, но требует установки дополнительных библиотек и может быть избыточным для простых проектов. **SDL2** — низкоуровневая библиотека, дающая полный контроль, однако её использование сопряжено с большим объёмом кода для инициализации и управления ресурсами. **Raylib** позиционируется как библиотека для обучения и прототипирования: она сочетает простоту использования, хорошую документацию и достаточную производительность. В отличие от GLFW (которая лишь создаёт окно и контекст OpenGL), Raylib предоставляет готовые функции для рисования примитивов, работы с текстурами и шрифтами, а также встроенные функции для работы со временем и векторами.

1.3. Выбор языка программирования и библиотеки

Для разработки был выбран язык **C++20** по следующим причинам:

- Высокая производительность и контроль над памятью, необходимые для обработки большого количества снарядов (до нескольких сотен одновременно).
- Поддержка современного стандарта C++20, включающего `std::format` для удобного форматирования строк интерфейса и `std::span`, `std::ranges` для работы с контейнерами.
- Кроссплатформенность и наличие качественных компиляторов (GCC, Clang, MSVC).

Библиотекой для графики, ввода и оконного управления выбрана **Raylib**

(версия 4.0 и выше). Обоснование выбора:

- Лицензия zlib/libpng, позволяющая свободное использование в любых проектах.
- Отсутствие внешних зависимостей (кроме системных библиотек OpenGL, GLFW, ALSA и т.п.).
- Простой и интуитивно понятный API: для отрисовки спрайта достаточно одной функции `DrawTexturePro()`, а для обработки клавиш — `IsKeyDown()`.
- Встроенная поддержка работы со временем (`GetFrameTime()`) и математических функций для работы с векторами.
- Активное сообщество и хорошая документация.

1.4. Требования к разрабатываемой игре

На основе анализа жанра и возможностей инструментов были сформулированы следующие функциональные и нефункциональные требования.

Функциональные требования:

1. Игра должна содержать меню выбора уровня с возможностью переключения между доступными уровнями.
2. Игрок управляет персонажем с помощью клавиш WASD или стрелок; скорость движения нормализована для равномерного перемещения по диагонали.
3. Игрок имеет показатель здоровья (HP); при столкновении со снарядом HP уменьшается, а игрок становится неуязвимым на короткое время (0,5 с).
4. Снаряды представляют собой круги, движущиеся с постоянной скоростью по заданной траектории. При выходе за границы игрового мира снаряды удаляются.
5. Уровни описываются через временные события (атаки), которые порождают снаряды. События могут создавать новые события (например, взрыв бомбы).
6. После завершения всех событий и исчезновения всех снарядов уровень считается пройденным, игра переходит в состояние `LEVEL_COMPLETED`.
7. При падении здоровья до 0 игрок проигрывает, состояние игры — `GAME_OVER`.

Нефункциональные требования:

- Частота кадров должна быть не менее 60 FPS при 300 активных снарядах на современном оборудовании.
- Исходный код должен быть структурирован, разделён на логические модули (`game.cpp`, `render.cpp`, `main.cpp`).
- Сборка должна осуществляться с помощью системы CMake или Premake, поддерживаться в среде Linux.
- Интерфейс игры должен быть адаптивным к изменению размера окна.

1.5. Постановка задачи

На основе сформулированных требований необходимо разработать программную систему, реализующую игру в жанре bullet hell. В рамках курсовой работы требуется:

1. Спроектировать объектную модель, включающую классы `Player`, `Projectile`, `AttackEvent` и перечисление состояний игры (`GameState`).
2. Реализовать игровой цикл, поддерживающий переходы между состояниями `MENU`, `PLAYING`, `LEVEL_COMPLETED`, `GAME_OVER`.
3. Разработать механику движения игрока с нормализацией скорости и ограничением границами мира.
4. Реализовать систему событий, позволяющую задавать атаки в виде функций с привязкой к абсолютному времени уровня.
5. Создать библиотеку вспомогательных функций для часто используемых паттернов атак: `add_beam_events`, `spawn_circular_burst`, `spawn_bullet_line`, `add_fan_attack`.
6. Разработать два демонстрационных уровня: «Beginning» (обучающий) и «Way» (с более сложными паттернами).
7. Реализовать отрисовку с преобразованием мировых координат в экранные (с сохранением пропорций и центрированием).
8. Подготовить инструкцию по сборке и запуску на платформе Linux.

Решение перечисленных задач позволит получить работающий прототип игры, демонстрирующий основные принципы организации игровой логики на C++ с использованием Raylib.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ

2.1. Общая структура приложения

Приложение имеет три основных состояния, определяемых перечислением `GameState`:

- `MENU` — отображение меню выбора уровня, ожидание ввода пользователя.
- `PLAYING` — активный игровой процесс: обновление физики, снарядов, событий уровня, отрисовка.
- `LEVEL_COMPLETED` и `GAME_OVER` — состояния, из которых возможен только возврат в меню.

Главный цикл (файл `main.cpp`) на каждом кадре выполняет:

1. Обработку ввода (в зависимости от состояния).
2. Обновление игровой логики (`update_level`, `update_projectiles`, движение игрока).
3. Отрисовку через `render_scene`.

Также используется глобальная переменная `debug_mode`, которая влияет на отображение дополнительной информации (хитбоксы, FPS, координаты) и видимость отладочных уровней в меню.

2.2. Объектная модель

Разработаны следующие основные сущности.

Игрок (`struct Player`):

Листинг 2.1. Структура `Player` из `game.h`

```
1 struct Player {  
2     Vector2 pos;  
3     float speed;  
4     float width, height;  
5     int health;  
6     bool immortal;  
7     float invincible_until;  
8     int hit_count;  
9     bool hit(int damage);  
10 };
```

Метод `hit(int damage)` уменьшает здоровье, если не включён режим бессмертия, и возвращает `true`, если здоровье стало ≤ 0 (игрок погиб). Неуязвимость реализуется через сравнение `invincible_until` с текущим временем уровня.

Снаряд (struct Projectile):

Листинг 2.2. Структура Projectile из game.h

```
1 struct Projectile {
2     Vector2 pos;
3     Vector2 vel;
4     float r;
5 };
```

Все снаряды хранятся в `std::vector<Projectile>`. Такое решение обеспечивает эффективное добавление/удаление и обход элементов.

Событие атаки (AttackEvent):

Листинг 2.3. Структура AttackEvent из game.h

```
1 struct AttackEvent {
2     float time;
3     std::function<void(float dt, std::vector<Projectile
4         >&, Player&)> action;
5 };
```

Поле `time` — абсолютное время уровня (в секундах), когда должно произойти событие. `action` — функция, которая получает сдвиг времени `dt` (для точного позиционирования пуля), список снарядов и ссылку на игрока. Использование `std::function` позволяет передавать лямбда-выражения и именованные функции, что даёт большую гибкость.

2.3. Система управления игроком

Функция `move_player_wasd(Player& player)` обрабатывает ввод с клавиатуры. Алгоритм:

1. Получить вектор направления от нажатых клавиш (WASD или стрелки).
2. Нормализовать вектор, если он не нулевой (чтобы скорость по диагонали не была выше, чем по осям).

3. Обновить позицию: `pos += move * speed * dt`.
4. Ограничить позицию границами мира `[0, WORLD_SIZE]`.

Нормализация выполняется с помощью `Vector2Normalize()` из Raylib. Это стандартный приём для обеспечения равномерной скорости во всех направлениях.

2.4. Система снарядов и коллизий

Обновление снарядов происходит в функции `update_projectiles`:

1. Для каждого снаряда обновляется позиция на основе вектора скорости: `pos += vel * dt`.
2. Проверка столкновения с игроком с помощью функции `circle_rect_collision`.
3. При столкновении — увеличение счётчика попаданий и перемещение снаряда за границы мира (помечается на удаление).
4. Удаление снарядов, вышедших за пределы `[0, WORLD_SIZE]` по любой оси (используется `std::remove_if`).
5. Если есть попадания и неуязвимость прошла (`invincible_until < level_time`), вызывается `player.hit(1)` и устанавливается `invincible_until = level_time + 0.5f`.
6. Если игрок погиб, состояние игры переключается в `GAME_OVER`.

Функция проверки столкновения `circle_rect_collision` реализует алгоритм «круг–прямоугольник» через вычисление ближайшей точки прямоугольника к центру круга, что даёт точный результат.

2.5. Система событий и планировщик атак

Управление уровнями централизовано в функции `update_level`. Для уровней «Beginning» и «Way» используется событийная модель:

- Статический вектор `events` хранит все `AttackEvent` для текущего уровня.
- Статическая переменная `next_event` — индекс следующего необработанного события.
- При сбросе уровня (`reset_level == true`) вызывается функция инициализации (`level_beginning_init` или `level_way_init`), которая

заполняет `events` и сортирует их по времени.

- В каждом кадре, пока `next_event < events.size()` и `level_time >= events[next_event].time`, событие выполняется (с передачей `dt = level_time - events[next_event].time`), и `next_event` увеличивается.
- Уровень считается пройденным, когда все события обработаны и вектор `projectiles` пуст.

Для удобства создания типовых паттернов атак реализованы вспомогательные функции:

- `add_beam_events` — создаёт последовательность пуль (луч) из одной точки.
- `spawn_bullet_line` — создаёт линию пуль (одновременно).
- `spawn_circular_burst` — круговой залп из центра.
- `add_beam_burst` — множество лучей из центра (взрыв лучами).
- `add_fan_attack` — веер из нескольких лучей.
- `spawn_targeted_bullet` — прицельный снаряд.

Эти функции вызываются внутри `level_beginning_init` и `level_way_init`, что делает описание уровней компактным и наглядным.

2.6. Рендеринг и преобразование координат

Отрисовка выполняется в файле `render.cpp`. Поскольку игровой мир имеет фиксированный размер `WORLD_SIZE` (1024 пикселя), а окно может быть произвольного размера, необходимо преобразование мировых координат в экранные с сохранением пропорций.

Функция `world_to_screen(Vector2 world_pos, int screen_w, int screen_h)`:

1. Вычисляет масштаб: `scale_x = screen_w / WORLD_SIZE`, `scale_y = screen_h / WORLD_SIZE`, выбирает минимальный `scale`.
2. Размер игровой области: `view_w = WORLD_SIZE * scale`, `view_h = WORLD_SIZE * scale`.
3. Смещение для центрирования: `offset_x = (screen_w - view_w) / 2`, `offset_y = (screen_h - view_h) / 2` (прижато к правому

нижнему углу для удобства отображения информации).

4. Возвращает экранные координаты: $(\text{offset_x} + \text{world_pos.x} * \text{scale}, \text{offset_y} + \text{world_pos.y} * \text{scale})$.

Для отрисовки спрайта игрока используется `DrawTexturePro`, которая принимает исходный прямоугольник текстуры и целевой прямоугольник на экране. Размер целевого прямоугольника вычисляется через `world_to_screen(player.width)` и `world_to_screen(player.height)`. Это позволяет спрайту масштабироваться вместе с окном.

Хитбокс игрока (прямоугольник) отображается только в режиме отладки. Для этого используется `DrawRectangleLinesEx` с координатами, полученными преобразованием из мировых в экранные.

ГЛАВА 3. РЕАЛИЗАЦИЯ ИГРОВЫХ МЕХАНИК И УРОВНЕЙ

3.1. Реализация базовых механик

3.1.1. Движение игрока

Функция `move_player_wasd` считывает состояние клавиш WASD или стрелок, формирует вектор направления и нормализует его перед применением скорости. Нормализация необходима для того, чтобы движение по диагонали не было быстрее движения по осям. После обновления позиции она ограничивается границами мира (`WORLD_SIZE`).

Листинг 3.4. `move_player_wasd` (упрощённо)

```
1 void move_player_wasd(Player& player) {
2     float dt = GetFrameTime();
3     Vector2 move = {0, 0};
4     if (IsKeyDown(KEY_W) || IsKeyDown(KEY_UP)) move.y
5         -= 1;
6     if (IsKeyDown(KEY_S) || IsKeyDown(KEY_DOWN)) move.y
7         += 1;
8     if (IsKeyDown(KEY_A) || IsKeyDown(KEY_LEFT)) move.x
9         -= 1;
10    if (IsKeyDown(KEY_D) || IsKeyDown(KEY_RIGHT)) move.
11        x += 1;
12    if (move.x != 0 || move.y != 0)
13        move = Vector2Normalize(move);
14    player.pos.x += move.x * player.speed * dt;
15    player.pos.y += move.y * player.speed * dt;
16 }
```

3.1.2. Коллизии и неуязвимость

Проверка столкновения снаряда (круг) с игроком (прямоугольник) реализована в функции `circle_rect_collision`. Она использует оптимизированный алгоритм: сначала проверяются расстояния по осям, затем, при необходимости, угол прямоугольника.

Функция `update_projectiles` обрабатывает движение всех снарядов, проверяет столкновения и управляет неуязвимостью. Если игрок получает урон, его здоровье уменьшается, а `invincible_until` устанавливается на `level_time + 0.5f`. В течение этого времени повторные попадания не наносят урон.

Листинг 3.5. Фрагмент `update_projectiles`

```
1  bool dead = false;
2  if (hit_count > 0 && player.invincible_until <
    level_time) {
3      dead = player.hit(1);
4      player.invincible_until = level_time + 0.5f;
5  }
6  if (dead) game_state = GameState::GAME_OVER;
```

Для удаления снарядов, вышедших за пределы мира, используется идиома `std::remove_if` с последующим `erase`. Это обеспечивает линейную сложность и безопасное удаление.

3.2. Реализация системы событий и планировщика

3.2.1. Структура `AttackEvent`

Событие атаки привязано к времени, прошедшему с начала уровня, и хранит функцию, которая будет вызвана в момент срабатывания. Благодаря `std::function` можно передавать как лямбды, так и именованные функции.

3.2.2. Планировщик в `update_level`

Для событийных уровней (`BEGINNING`, `WAY`) используется статический вектор `events`, заполняемый при инициализации уровня. Сортировка событий по времени (функция `sort_attack_events`) необходима, чтобы выполнять атаки последовательно.

Листинг 3.6. Цикл обработки событий

```
1  while (next_event < events.size() && level_time >=
    events[next_event].time) {
2      float dt = level_time - events[next_event].time;
```



```

3     events[next_event].action(dt, projectiles, player);
4     next_event++;
5 }

```

Уровень считается завершённым, когда все события обработаны (`next_event == events.size()`) и вектор `projectiles` пуст.

3.2.3. Вспомогательные функции для генерации атак

Для сокращения дублирования кода созданы функции:

- `add_beam_events` — серия пуль вдоль луча,
- `spawn_bullet_line` — одновременная линия,
- `spawn_circular_burst` — круговой разлёт,
- `add_beam_burst` — взрыв лучами,
- `add_fan_attack` — веер лучей,
- `spawn_targeted_bullet` — прицельный снаряд.

Эти функции используют только стандартные средства Raylib и C++ и не зависят от глобального состояния.

3.3. Создание уровня «Beginning»

Уровень «Beginning» (инициализация в `level_beginning_init`) предназначен для демонстрации основных механик и обучения игрока. Последовательность атак:

1. Две плотные линии пуль, движущиеся сверху вниз (с разрывом в центре).
2. Широкая линия с щелями.
3. Бомба, падающая в центр, после взрыва — круговая волна.
4. Веер из трёх лучей (центр, лево, право).
5. Встречные горизонтальные лучи (слева направо и справа налево).
6. Прицельная пуля, выпущенная из случайной точки вверх.
7. Веер из пяти лучей.
8. Вторая бомба с комбинированным взрывом (лучи + линия).

Каждая атака добавляется в вектор `events` с указанием времени старта. Для бомб используется двухэтапное событие: сначала создание бомбы, затем через расчётное время — её взрыв с порождением новых событий. В момент взрыва бомба удаляется из списка снарядов поиском.

3.4. Создание уровня «Way»

Уровень «Way» (функция `level_way_init`) содержит более сложные паттерны:

1. Два расходящихся луча из центра верхней границы.
2. Широкая линия сверху.
3. Две бомбы, падающие одновременно: одна даёт круговую волну, другая — взрыв лучами.
4. Веер из пяти лучей, горизонтальный луч и линия снизу.
5. Прицельная пуля и встречная линия снизу.
6. Два встречных веера (слева и справа).
7. Каскадная бомба: после падения — круговая волна и диагональные пули, которые при пересечении в центре порождают вторую круговую волну.

3.5. Режим отладки и вспомогательные функции

Режим отладки включается клавишей F3 и управляется глобальной переменной `debug_mode`. В этом режиме:

- На экран выводятся FPS, координаты игрока, счётчик попаданий, текущее время уровня.
- Отображается хитбокс игрока (прямоугольник).
- В меню становятся видимыми тестовые уровни (`EMPTY`, `TEST`, `TEST_HP`).

Для преобразования мировых координат в экранные с сохранением пропорций реализованы функции `world_to_screen` (для точки и для размера). Отрисовка спрайта игрока использует `DrawTexturePro` с автоматическим масштабированием, чтобы сохранить пропорции текстуры при изменении размера окна.

3.6. Основные трудности и их решение

В ходе разработки возникло несколько проблем, которые затем были решены. В таблице 3.1 перечислены основные трудности и способы их преодоления.

Трудности и их решения

Проблема	Решение
Неправильная обработка коллизий при не квадратном хитбоксе	Замена функции расчёта коллизий.
Неправильная сортировка событий при динамическом добавлении	Вынесение сортировки в отдельную функцию, вызов после добавления новых событий
Завершение уровня при паузе отладки (level_time скачком)	Ограничение dt сверху (0.5 сек)
Статическое состояние уровней не сбрасывалось при рестарте	Добавлен флаг reset_level, обнуление static-переменных
Хитбокс игрока не соответствовал текстуре	Обрезана текстура

Дополнительные сложности:

- **Удаление бомб при взрыве:** в обработчике взрыва необходимо идентифицировать бомбу в векторе `projectiles`. Использована проверка по радиусу и приближительному положению.
- **Нормализация движения:** без нормализации скорость по диагонали в $\sqrt{2}$ раз выше, что нарушало баланс. Исправляется вызовом `Vector2Normalize`.
- **Преобразование координат:** изначально игровая область неправильно центрировалась при изменении размера окна. Добавлено вычисление смещения `offset_x, offset_y`.

Эти исправления позволили получить стабильную версию игры.

ГЛАВА 4. ТЕСТИРОВАНИЕ И ОТЛАДКА

4.1. Цели и подходы к тестированию

В процессе разработки проводилось функциональное тестирование игровых механик, а также тестирование производительности и стабильности при работе с большим количеством снарядов. Основные цели:

- проверка корректности движения игрока и обработки ввода;
- проверка коллизий и системы неуязвимости;
- проверка работы планировщика событий и правильности последовательности атак;
- проверка удаления снарядов за границами мира;
- оценка производительности (частоты кадров) при максимальной нагрузке.

Для изолированной проверки отдельных механик были созданы специальные тестовые уровни, которые становятся видимыми в меню при включённом режиме отладки (клавиша F3).

4.2. Тестовые уровни

4.2.1. Уровень TEST

Уровень TEST предназначен для проверки базовых возможностей системы снарядов. Он генерирует три типа снарядов:

- две движущиеся пули (разного радиуса), вылетающие из точки (700,700) вверх;
- одну статическую пулю большого радиуса (для проверки коллизий в статике).

На этом уровне игроку даётся бессмертие (`player.immortal = true`), что позволяет безопасно проверять корректность отрисовки и коллизий без риска проигрыша.

4.2.2. Уровень TEST_HP

Уровень TEST_HP предназначен для проверки системы здоровья и неуязвимости. На нём с интервалом 1 секунду создаётся одна движущаяся пуля. Игрок имеет 3 единицы здоровья и не имеет бессмертия. Этот уровень позволяет убедиться, что:

- при попадании здоровье уменьшается на 1;
- при достижении здоровья 0 игра переходит в состояние `GAME_OVER`.

4.2.3. Уровень EMPTY

Пустой уровень, на котором не создаётся ни одного снаряда. Используется для проверки работы игрового цикла в отсутствие атак, а также для тестирования интерфейса выбора уровней и перехода между состояниями.

4.3. Тестирование планировщика событий

Для проверки корректности работы событийной модели на уровнях «Beginning» и «Way» использовались следующие методы:

- визуальный контроль последовательности атак (соответствие задуманным паттернам);
- проверка времени появления каждого события с использованием отображения `level_time` в режиме отладки;
- проверка динамического добавления событий (например, при взрыве бомбы на уровне «Way»);
- проверка завершения уровня: после обработки всех событий и исчезновения всех снарядов игра должна перейти в состояние `LEVEL_COMPLETED`.

В процессе тестирования были выявлены и исправлены ошибки, описанные в таблице 3.1 главы 3. В частности, проблема сортировки событий после динамического добавления была решена введением функции `sort_attack_events`, вызываемой после каждого добавления новых событий.

4.4. Тестирование производительности

Одним из ключевых требований к игре в жанре *bullet hell* является поддержание стабильной частоты кадров (не менее 60 FPS) при большом количестве одновременно активных снарядов. В процессе тестирования использовался встроенный счётчик FPS, доступный в режиме отладки. Кроме того, для оценки нагрузки на систему был добавлен счётчик активных снарядов.

Результаты тестирования на уровне «Way» показали, что на оборудовании с процессором Intel Core i5 и интегрированной графикой частота кадров

оставалась не ниже 60 FPS при количестве снарядов до 300.

4.5. Режим отладки как инструмент тестирования

Режим отладки (включается клавишей F3) предоставляет разработчику следующие возможности:

- отображение FPS и количества активных снарядов;
- отображение точных координат игрока и текущего времени уровня;
- визуализация хитбоксов: для игрока рисуется прямоугольник;
- отображение счётчика попаданий (сколько раз игрок столкнулся со снарядами за уровень);
- доступ к тестовым уровням в меню.

Использование режима отладки позволяет быстро выявлять ошибки коллизий, проверять точное время срабатывания событий и анализировать производительность.

4.6. Результаты тестирования

По итогам тестирования можно сделать следующие выводы:

1. Все базовые механики (движение, коллизии, неуязвимость, здоровье) работают корректно.
2. Событийная система обеспечивает точное воспроизведение запланированных атак на уровнях «Beginning» и «Way».
3. Производительность достаточна для комфортной игры на современном оборудовании.
4. Режим отладки и тестовые уровни упрощают процесс выявления и исправления ошибок.
5. Выявленные в процессе разработки трудности успешно преодолены.

ГЛАВА 5. СБОРКА И ЗАПУСК

5.1. Требования к системе

Для сборки и запуска игры необходимо следующее программное обеспечение:

- Операционная система: Linux (Debian-based). Возможна сборка на других платформах, но программа ориентирована для Linux.
- Компилятор: GCC 10 или новее (поддержка C++20).
- Система сборки: Premake5.
- Библиотеки: Raylib (версия 4.0 или новее), а также системные зависимости: OpenGL, GLFW, X11.
- Рекомендуемый редактор: VSCode (с расширениями C/C++).

5.2. Установка зависимостей

Перед сборкой необходимо установить компилятор, утилиты сборки и библиотеку Raylib. В дистрибутивах на основе Debian/Ubuntu это делается командами:

Листинг 5.7. Установка системных зависимостей

```
1 sudo apt update
2 sudo apt install build-essential git
3 sudo apt install libx11-dev libasound2-dev libxrandr-
  dev libxi-dev libgl1-mesa-dev libglu1-mesa-dev
  libxcursor-dev libxinerama-dev
```

5.3. Получение исходного кода

Исходный код проекта размещён в открытом репозитории GitHub. Для клонирования выполните:

Листинг 5.8. Клонирование репозитория

```
1 git clone https://github.com/kovalev2p/
  projectile_game.git --recurse-submodules && cd
  project-name
```

Структура проекта соответствует шаблону `raylib-quickstart`:

- `src/` — исходные файлы (`main.cpp`, `game.cpp`, `render.cpp` и заголовочные файлы);
- `build/` — каталог, содержащий `premake5.lua` и скрипты сборки;
- `external/` — автоматически создаётся при сборке, содержит загруженную Raylib;
- `bin/` — выходные файлы после компиляции.

5.4. Сборка с использованием Premake

Проект использует Premake для генерации Makefile-ов. Файл `premake5.lua` настроен таким образом, что при первом запуске автоматически загружает архив Raylib с GitHub, распаковывает его в папку `external/` и компилирует статическую библиотеку.

Для сборки необходимо выполнить следующие команды:

Листинг 5.9. Сборка через Premake

```
1  cd build
2  ./premake5 gmake
3  cd ..
4  make
```

После успешной компиляции исполняемый файл появится в каталоге `bin/`:

5.5. Запуск игры

Для запуска необходимо перейти в каталог `bin/Release/` (или `bin/Debug/`) и запустить исполняемый файл программы.

5.6. Управление в игре

В таблице 5.1 приведены основные клавиши управления.

Управление в игре

Клавиша(и)	Действие
W / Up	Движение вверх
S / Down	Движение вниз
A / Left	Движение влево
D / Right	Движение вправо
Enter / Space	Выбор уровня в меню / начало игры
Escape	Выход из игры (в меню) / возврат в меню (в игре)
F3	Включение/выключение режима отладки

В режиме отладки на экран выводятся дополнительная информация (FPS, координаты игрока, счётчик попаданий, текущее время уровня) и хитбоксы. Также в меню становятся доступными тестовые уровни.

5.7. Возможные проблемы и их решение

- **Ошибка «X11/Xlib.h not found»:** Premake проверяет наличие заголовочных файлов X11. Если они отсутствуют, сборка прерывается с сообщением об ошибке. Решение: установить пакет `libx11-dev` (и сопутствующие) как указано в разделе 5.2.
- **Raylib не скачивается автоматически:** Проверьте подключение к интернету. При медленном соединении можно скачать архив вручную с GitHub (<https://github.com/raysan5/raylib/archive/refs/heads/master.zip>) и поместить его в папку `external/` перед запуском Premake.
- **Ошибка компиляции из-за стандарта C++20:** Убедитесь, что компилятор поддерживает C++20 (GCC 10+ или Clang 12+).
- **Низкая производительность:** Отключите режим отладки (F3) и закройте фоновые приложения. Для слабых видеокарт можно уменьшить размер окна или изменить версию OpenGL на более раннюю.
- **Пути к ресурсам:** Игра ожидает текстуру `wabbit_alpha.png` в каталоге `resources/`. Убедитесь, что этот файл присутствует и находится в рабочем каталоге при запуске. Premake при сборке копирует ресурсы в папку `bin/`.

ЗАКЛЮЧЕНИЕ